simpleXRPL — Technical Design Document

Related: simpleXRPL - PRD_v3, simpleXRPL - API Surface v2, custody.js - a JS/TS SDK for interacting with the Ripple Custody API

1. Context & Scope

`simpleXRPL` is a high-level TypeScript SDK for the XRP Ledger, scoped to the institutional / Web2 audience that the PRD targets. Its goal is to collapse the distance between a business intent ("transfer this token", "freeze that holder", "issue this credential") and the XRPL transactions that realize it, without forcing the caller to learn the transactor specific details, the flag combinations, or the signing-backend differences.

1.1 In scope (v1)

- Write methods across verticals (classes in API Surface), dispatched to one of three signing backends:
  - **Local signing** via `xrpl.js` `Wallet.sign()` + `client.submit()`.
  - **Ripple Custody** via Custody REST API v1 (`v0_CreateTransactionOrder` for natively supported transactors; `v0_SignManifest` with `content.type: "Unsafe"` for transactors Custody does not yet model).
  - **Palisade Custody** via Palisade REST API (`TransactionsService_*` for natively supported transactors; `RawTransaction` / `SignPlaintext` for transactors Palisade does not natively model).
- A `Custodian` abstraction; each custodian instance owns its own primary account/wallet and discovers the rest of its accounts at construction.
- Mixed-signer clients (per- `Account` signer assignment).
- Reads do not require a signer (they go through `xrpl.js` directly); a client with no signer configured can still query the ledger; signing methods throw `NoSignerError`.
- Sync `submitAndWait`-style execution by default; opt-in async handle.
- A typed, discriminated-union response shape.
- Pre-flight validation (intent shape, amount precision, alias resolution).
- A documented but not-public-contract `customProperties` block stamped on every Custody intent.
- **Read-only observation of custodian governance multi-approval**: `client.intent.status(id)` and `client.intent.await(id)` resume blocking on a Custody/Palisade intent the SDK previously created.

1.2 Out of scope (v1)

- Custodian providers other than Ripple Custody and Palisade. The `Custodian` interface is shaped to accept additional providers (Fireblocks, BitGo, Fordefi, Anchorage), but they don't ship in v1.
- Generate intent-author credentials, Palisade API keys, or seeds. Those are caller's responsibilities.
- **Approver-side primitives**. The SDK does not expose `intent.approve`, `intent.reject`, `intent.listPending`, or any policy admin surface. Approval is performed by a different human, via the custodian's own UI or API, under different credentials (the custodian enforces dual control) via a separate "approver app".
- **XRPL protocol multisign** — both configuring `SignerList` and assembling `Signers[]` multi-signed transactions. Dropped from v1 to reduce complexity: the institutional "no single party can move funds" need is met by custodian governance multi-approval (§10.4), which most clients use anyway. On-ledger multisign may be revisited in a later version.

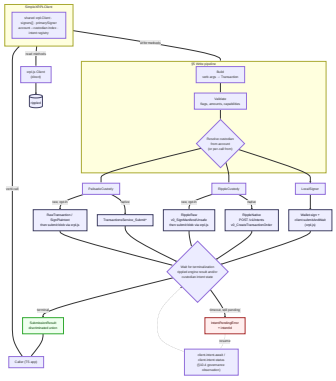
1.3 Terminology

- **Verb**: an SDK method expressing a business action (`Token.transfer`, `IOU.issue`). The public surface callers use.
- **Vertical**: a class grouping related verbs: `Token`, `IOU`, `Credential`, `Domain`, `Account`, plus the `XRP` helper.
- **Transactor**: the XRPL transaction type a verb maps to (`Payment`, `TrustSet`, ...). One verb can expand into several.
- **Custodian / adapter**: a signing backend: `LocalSigner`, `RippleCustody`, `PalisadeCustody`. Each *adapts* a transaction to its backend's API (§4, §7).
- **Intent**: the custodian-side request a transactor becomes (Custody `v0_CreateTransactionOrder`, Palisade `Submit*`). Custodians review/approve/sign intents; the SDK only proposes and observes them.
- **Path**: the concrete route a write takes once dispatched: Local, RippleNative, RippleRaw, PalisadeNative, or PalisadeRaw (§6).

2. Architecture Overview

A caller invokes a verb on the client. **Reads** go straight to rippled through `xrpl.js`. **Writes** flow through the pipeline (§5): build the `xrpl.js` transaction, validate it, and resolve which custodian owns the sending account, then dispatch to one of five paths (Local, RippleNative, RippleRaw, PalisadeNative, PalisadeRaw). All paths converge on one wait step that returns a

typed `SubmissionResult`, or, when a custodian approval is still pending past the timeout, an `IntentPendingError` the caller can resume later (§10).



3. Configuration & Initialization Reference

3.1 `SimpleXRPL.init`

`SimpleXRPL` is a namespace; `SimpleXRPL.init(...)` is the only entry point and returns an instance of `SimpleXRPLClient` (§4). Each custodian's `create()` / `fromEnv()` is async (authentication + account discovery happen up front, before the first verb call); the runtime client is never constructed via `new`.

```
await SimpleXRPL.init({
  rippledUrl: string,              // ws(s):// or http(s)://
  faucetUrl?: string,              // testnet only
  signers?: Custodian[],           // pre-constructed
  primarySigner?: Custodian,       // defaults to signers[0]
  // other opts will be added upon implementation
});
```

Discovery rules at init:

1. If `signers` is provided, init authenticates none of them (each was already authenticated at its own construction) and validates them: no duplicate `kind` + same tenant id, no r-address appearing under two custodians (`AmbiguousAccountError` if so).
2. If `signers` is omitted and any `XRPL_*_SEED` env var is present, `LocalSigner.fromEnv()` is constructed and used as the single custodian.
3. If `signers` is omitted and no env seeds are present, the client starts with no signer (§9.3). Reads still work; write calls throw `NoSignerError`.

3.2 `LocalSigner`

```
LocalSigner.fromSeed(seed: string);
```

```
LocalSigner.fromMnemonic(phrase: string, derivation?: string); // Removing this since marked as deprecated on xrpl.js
LocalSigner.create({ wallets, primary? });
LocalSigner.fromEnv({ primary? });
```

Values will be read from `.env`; if primary is not specified, first match in scanning order is primary. Seed and mnemonic sources are the caller's responsibility.

3.3 `RippleCustody`

Env var	Maps to
`RIPPLE_CUSTODY_GATEWAY_URL`	`gatewayUrl`
`RIPPLE_CUSTODY_AUTH_SIGNING_KEY`	`auth.signingKey`
`RIPPLE_CUSTODY_AUTH_PUBLIC_KEY`	`auth.publicKey` (optional)
`RIPPLE_CUSTODY_AUTH_TOKEN_URL`	`auth.tokenUrl`
`RIPPLE_CUSTODY_DOMAIN_ID`	`domainId`

`RIPPLE_CUSTODY_AUTH_SIGNING_KEY` is the intent-author **private key** (either the PEM contents or a path to a `.pem` file), the ID key registered with Custody at onboarding, which signs both the JWT challenge and every canonicalized intent body. Its algorithm (`secp256k1` / `ed25519` / `secp256r1`) is **auto-detected** from the key, so the caller doesn't declare it. For production, source it from a secret manager rather than a `.pem` on disk (§12.2).

`auth.publicKey` is the matching public key in **Base64**, the exact value registered with Custody. It is **optional**: if omitted, the SDK derives it from the private key (via Node's `crypto`, which covers all three algorithms), but supplying it explicitly avoids any mismatch with the registered encoding. Note this is the API / ID key — *not* the XRPL ledger-signing key, which never leaves the vault.

```
const ripple = await RippleCustody.fromEnv({ primary: "rTreasury..." });
```

`primary` (the primary account r-address) is always required and is validated against the discovered account list during `fromEnv` / `create`.

Optional: `defaultFee?: { priority?: "low" | "medium" | "high"; maxFeeDrops?: string }` sets this custodian's house fee intent (§7). It falls back to `Priority: Low` plus a safety cap and is overridable per call, so the Custody-required `feeStrategy` is supplied without the caller passing `fee` each time.

Optional: `defaultDryRun?: boolean` (default `false`) pre-flights every write through Custody's dry-run (§5.2) before the real intent.

3.4 `PalisadeCustody`

Env var	Maps to
`PALISADE_BASE_URL`	`baseUrl`
`PALISADE_API_KEY`	`apiKey`
`PALISADE_ORG_ID`	`orgId`
`PALISADE_ALLOW_RAW_SIGNING`	`allowRawSigning`

```
const palisade = await PalisadeCustody.fromEnv({
  primary: { vaultId: "v_...", walletId: "w_..." },
});
```

The `primary` pair is always required.

Optional: `defaultFee?: { maxFeeDrops?: string }` sets this custodian's house max-fee cap, mapped to Palisade's `networkFee` (drops→XRP decimal). Palisade has no submit-time priority tier, so only the cap applies; otherwise the fee is Palisade-managed (§7).

3.5 Logger and redaction

The default `Logger` interface mirrors a minimal subset of `pino` / `bunyan` (`debug`, `info`, `warn`, `error`). Implementations are passed field-redaction lists for known-sensitive keys: `signingKey`, `apiKey`, `seed`, `mnemonic`, `access_token`, `refresh_token`, etc.

4. Internal Domain Model

The model is intentionally thin. It separates **identity** (who can sign for what), **intent** (what the caller wants), and **transactor** (the XRPL operation produced).

```
SimpleXRPLClient
 ├── network: { rippledUrl, faucetUrl? }
 ├── signers: Custodian[]                  // 0..N, registered at init
 ├── primarySigner: Custodian | undefined  // defaults to signers[0]
 ├── accounts: Map<string, Account>    // discovered from custodians
```

```
 └── intent: IntentInspector              // status/await

Custodian  (interface)
 ├── kind: "local" | "ripple-custody" | "palisade-custody"
 ├── primary: AccountRef                  // owns its primary
 ├── listAccounts(): Promise<Account[]>   // populated at construction
 ├── capabilities(): SignerCapabilities
 ├── sign(tx, ctx): Promise<SignedEnvelope>
 ├── submitAndWait(tx, ctx): Promise<SubmissionResult>
 └── submitAsync(tx, ctx): Promise<SubmissionHandle>

LocalSigner     implements Custodian   // wraps xrpl.Wallet(s) + client.submit
RippleCustody   implements Custodian   // wraps Custody REST API (v1)
PalisadeCustody implements Custodian   // wraps Palisade REST API

Account
 ├── address: string                      // r-address (the canonical key)
 ├── alias?: string                       // custodian-side alias if any
 ├── custodianRef?: string | {vaultId,walletId} // opaque native id; only the owning
custodian reads it
 ├── signer: Custodian                    // back-reference to the parent
 └── metadata: { kind?, tags? }           // optional, advisory only
```

Key properties:

- `Custodian` **is the unit of configuration.** Each custodian instance carries its own credentials and primary account. The client composes one or more custodians at init.
- **The** `Custodian` **implementations are the "adapters."** `LocalSigner`, `RippleCustody`, and `PalisadeCustody` each adapt the canonical `xrpl.js` `Transaction` to one backend's API and submission flow (the per-backend mapping is §7).
- **Accounts are discovered, not registered.** Each `Custodian` returns its full account list from `listAccounts()` at construction. The client flattens these into a single `accounts` map keyed by r-address. The primary signer's primary account is used as the default when a verb is called without an explicit `from`.
- `Account.signer` **is a back-reference to the discovering custodian.** Dispatch consults `account.signer.capabilities()` at the moment a method runs, not at client construction.
- **One uniform** `Account`**, keyed by r-address, no per-custodian subclasses.** The r-address is the canonical key the core and verticals use; the native identifier (`custodianRef`) is opaque to the core, a string for Custody (account UUID), a `{ vaultId, walletId }` pair for Palisade, absent for Local, and is read only by the owning `Custodian`. Per-custodian behavior lives in the `Custodian` implementation (the strategy), not in the `Account` record, so nothing downstream has to branch on account type.
- **No credentials per call.** Verticals (`Token`, `IOU`, etc.) accept an address, alias, or `AccountRef` and never accept seeds or API keys directly.
- **Identity is plural per custodian.** One `RippleCustody` backs all accounts in a Custody domain; one `PalisadeCustody` backs all wallets in a Palisade org; one `LocalSigner` can

carry multiple `xrpl.Wallet`s keyed by address.

- **No-signer mode is valid.** With `signers: []` (or omitted and no `XRPL_*_SEED` env var), the client still works for reads and intent inspection. Any write verb throws `NoSignerError` with a message that names the missing configuration knob.

Vertical class shape

Each vertical is a small class that takes the client at construction and exposes intent-level methods. Account selection follows the "primary-or-explicit" rule: omit `from` to use `primarySigner.primary`; pass `from` to override.

```
class Token {
  constructor(private client: SimpleXRPLClient) {}

  issue(
    params: TokenIssueParams,
    opts?: { from?: AccountSelector }
  ): Promise<SubmissionResult<TokenIssueOutput>>;

  transfer(
    params: { to: string; amount: TokenAmount },
    opts?: { from?: AccountSelector }
  ): Promise<SubmissionResult<PaymentOutput>>;
  // ...
}

// AccountSelector options:
//   "rTreasury..."                                  // address string
//   { address: "rTreasury..." }                     // explicit
//   { signer: ripple }                              // use that signer's primary
//   { signer: ripple, account: "rTreasury2..." }    // explicit pair
```

The vertical does not know whether the submission will go Local, RippleRaw, RippleCustody, or Palisade. It builds the intent, calls into the pipeline (§4), and returns the result. All path-awareness lives in the dispatcher and the `Custodian` implementations.

5. Pipeline

Every write method runs through the same six stages:

5.1 Build

The vertical translates the caller's business intent into one or more `xrpl.js` `Transaction` objects. This step populates only fields intrinsic to the intent (`TransactionType`, `Account`, `Destination`, `Amount`, `Flags`, transactor-specific fields). Network-derived fields (`Sequence`, `Fee`, `LastLedgerSequence`) are filled in stage 3.

For multi-step verbs (e.g. `IOU.issue`), Build produces an ordered list of `(Transaction, Account)` pairs. Each pair is dispatched independently.

5.2 Validate

Validation runs in two layers, in order:

1. **Protocol (`xrpl.js`).** `validate(tx)` is invoked on the Transaction produced by Build; it dispatches to the per-transactor validators (`validatePayment`, `validateTrustSet`, etc.) and covers both structural checks (required fields, types, enum membership) and protocol-level semantic checks (XRP vs `IssuedCurrencyAmount` compatibility, flag-combination rules such as `tfPartialPayment` requiring `SendMax`, etc.). Any thrown `ValidationError` is caught and re-wrapped as `IntentValidationError` so callers see a single error class.
2. **Capability (`simpleXRPL`).** Two sub-steps:
   a. *Lookup.* Resolve the source `Account` (primary, or `from.account` override) against the in-memory account-to-custodian index. An unknown account throws `UnknownAccountError`.
   b. *Match.* Check the Transaction type against the resolved `custodian.capabilities()`. If the transactor is not in `nativeOps` and `allowRaw` is false (or the transactor is not raw-signable), throw `SignerCapabilityError`. See §4 and §12.1.
3. **Dry-run (optional, custodian-side).** Off by default; enabled per call (`dryRun: true`) or as a **per-custodian default** (`defaultDryRun` on `RippleCustody`, §3.3). Where the backend supports it, the SDK submits the built intent to the custodian's dry-run endpoint — Ripple Custody's `POST /v1/intents/dry-run` (custody.js buildDryRun) — which runs the custodian's own schema + policy checks and returns a fee estimate **without creating a real intent** (no intent id consumed, no approvers notified). This catches what the local layers cannot: governance-policy rejection, custodian-side schema/field gaps, and fee/balance viability. It costs a network round-trip, hence opt-in. Local has no server-side dry-run; Palisade has no dry-run either. A failed dry-run throws `IntentValidationError` or surfaces the custodian's diagnostic, before any state-changing call is made.

5.3 Resolve

Fills in `Sequence`, `Fee`, `LastLedgerSequence`, and any version-dependent flags. For `LocalSigner` accounts this uses `xrpl.js` autofill against the connected rippled. For `RippleCustody` accounts on RippleNative, resolution is delegated to Custody (Custody owns the sending account and computes these itself); simpleXRPL passes the business-level fields only. For RippleRaw and for `PalisadeCustody` raw-signing fallbacks, simpleXRPL resolves fields itself because the custodian is signing an externally constructed blob. For `PalisadeCustody` native operations, resolution is delegated to Palisade.

5.4 Dispatch

The dispatch rule is documented in §6. In short: `LocalSigner` → Local; `RippleCustody` → RippleNative if the transactor is in the capability set, else RippleRaw if `allowRawSigning: true`, else throw; `PalisadeCustody` → native operation if supported, else raw-signing fallback if `allowRawSigning: true`, else throw.

5.5 Sign + submit

The `Custodian` implementation owns this stage:

- `LocalSigner` calls `Wallet.sign()` + `client.submit()`.
- `RippleCustody` RippleNative posts a `v0_CreateTransactionOrder` intent. RippleRaw posts a `v0_SignManifest` intent with the prepared blob, then submits the returned signed transaction to rippled via the shared `xrpl.js` client.
- `PalisadeCustody` native posts the appropriate `TransactionsService_Submit*` request (e.g. `SubmitOfferCancel`, `SubmitTrustSet`). Raw-signing fallback uses `RawTransaction` or `SignPlaintext` and submits the result via the shared `xrpl.js` client.

5.6 Wait + Wrap

By default the pipeline blocks until the transaction reaches a terminal state or a timeout. The result is wrapped in the discriminated union:

```
type SubmissionResult<T = unknown> =
  | { source: "rippled"; response: xrpl.TxResponse; intent: T }
  | { source: "custody"; response: CustodyTransactionResult; intent: T }
  | { source: "palisade"; response: PalisadeTransactionResult; intent: T };
```

The `intent` field carries any vertical-specific output (e.g. the `tokenID` produced by `Token.issue`) so callers don't need to parse the underlying response to get the canonical follow-up identifier. All three variants also expose a normalized `intentId?: string` (custodian intent ID when applicable) and `txHash?: string` (XRPL transaction hash once on-ledger) for convenience.

6. Dispatch Rule

For an `Account` discovered on a `SimpleXRPLClient`:

```
function dispatch(txIntent, account):
  signer = account.signer            // the parent Custodian
  transactor = txIntent.transactionType

  if signer.kind == "local":
    return Local

  if signer.kind == "ripple-custody":
```

```
    if transactor in signer.capabilities.nativeOps:
      return RippleNative
    if signer.config.allowRawSigning:
      return RippleRaw
    throw SignerCapabilityError(
      "RippleCustody for account '${account.address}' cannot sign " +
      "${transactor}. Either enable allowRawSigning: true or use " +
      "a different Custodian (LocalSigner, PalisadeCustody) for " +
      "this account."
    )

  if signer.kind == "palisade-custody":
    if transactor in signer.capabilities.nativeOps:
      return PalisadeNative
    if signer.config.allowRawSigning:
      return PalisadeRawSigning
    throw SignerCapabilityError(...)
```

For multi-step verbs (e.g. `IOU.issue`), the rule runs once per sub-transaction, against the account that signs that step. The orchestrator commits each step before starting the next, partial failure semantics are covered in §8 and §11.

7. Intent Construction & Payload Mapping

Once dispatch (§6) selects a path, the chosen **custodian adapter** (the backend's `Custodian` implementation; §4) translates the internal `xrpl.js` `Transaction` into that backend's request shape and submits it. This is a per-transactor translation, not a field copy, and it is **lossy in one direction**: each custodian's native operation schema is a curated *subset* of the XRPL transactor. The field-level mapping tracks each custodian's API and will move as those APIs evolve, so this section fixes the **design choices**, not the mapping details.

7.1 Design principles

- **One canonical internal model.** Verticals always build a standard `xrpl.js` `Transaction` (§5.1). Every adapter consumes that single object; no vertical ever constructs a Custody- or Palisade-specific payload. Adding a custodian means adding one adapter, not touching the verticals.
- **The adapter owns the wire shape.** Each `Custodian` implementation is solely responsible for mapping the `Transaction` to its backend and for any per-backend envelope, authentication, and canonicalization. The SDK core treats the produced request and response opaquely; §5.6 wraps the response in the discriminated union.
- **Who fills network fields depends on the path** (§5.3): on custodian-native paths the custodian owns sequence/fee/validity, so the SDK sends business-level fields only; on Local and raw paths the SDK autofills them itself.
- **Custodian types are generated, not hand-authored.** Each adapter maps `xrpl.js` types to types generated (via `openapi-typescript`) from that provider's spec — Custody's

`products/custody/@v1.35/api/reference/openapi.json` and Palisade's `products/wallet/api-docs/palisade-api/palisade-api.yaml`. The compiler then checks every mapping against the real contract, and regenerating after an API change surfaces drift as type errors — the mapping can't silently lie about what the server accepts. The only hand-written types are SDK-internal ones with no API counterpart. (The reference SDK mandates exactly this — see custody.js CLAUDE.md.)

- **Adapters depend on injected I/O ports.** Each adapter takes its network operations (submit-intent, poll-status, resolve-account) as injected ports - HTTP in production, in-memory fakes in tests - so the mapping and orchestration are exercised offline (§13.1). Cross reference custody.js xrpl.ports.ts.

7.2 Per-custodian payload mapping

Ripple Custody — native. The adapter maps the `Transaction` to a `v0_CreateTransactionOrder` operation (the per-transactor switch — see custody.js txToOperation), wraps it in Custody's signed-intent envelope (author identity, target domain, a client-supplied intent id, and the `customProperties` summary below — see xrpl.builders.ts), signs the canonicalized envelope with the intent-author key, and posts it.

Ripple Custody — raw (opt-in). Used only where Custody has no native operation for the transactor (§6). The SDK builds and autofills the whole transaction, has Custody sign the serialized form via `v0_SignManifest` + `Unsafe` (documented in `products/custody/@v1.35/api/payloads.md`), reassembles the signed blob, and submits it through the shared `xrpl.js` client. Cross reference: custody.js xrpl.service.ts (`rawSign` / manifest polling).

Palisade - native / raw. Native maps the `Transaction` to the matching `TransactionsService_Submit*` request (Palisade owns network fields and submission). Raw serializes the transaction, requests a sign-only signature, and submits the returned blob via `xrpl.js`.

7.3 Coverage gaps

Since each custodian's operation schema is a curated *subset* of the XRPL transactor, option with no native slot is never silently dropped: if `allowRawSigning` is enabled the adapter falls back to the raw path; if it's disabled the SDK throws `SignerCapabilityError` (caught pre-flight in Validate, §5.2, before any custodian-side intent is created), naming the unsupported option and the ways forward — enable `allowRawSigning`, drop the option, or use a Local/Palisade account. On Custody, for example, `Payment` has no `SendMax`/paths (no cross-currency path payments), `AccountSet` exposes only

`setFlag` / `clearFlag` / `transferRate`, and `OfferCreate` has no `Expiration` / `OfferSequence`.

Detecting an unsupported option and choosing reject-vs-raw is the adapter's main judgment call; the per-transactor contract tests (§13.1) exist to pin it. Everything else is deterministic one-to-one remapping.

7.4 Fee handling

Custody native takes a fee *strategy* plus a cap, not an exact `Fee` — so an exact-fee request can't be honored natively. Where `xrpl.js` sets an absolute fee, Custody takes either a priority tier (it derives the drops from network load) or an additive bump, both bounded by `maximumFee`:

```
// xrpl.js — exact fee, in drops
{ "TransactionType": "Payment", "Fee": "12", /* ... */ }

// Ripple Custody, option A — priority tier (Custody picks the drops, up to the cap)
"feeStrategy": { "type": "Priority", "priority": "High" },  // High | Medium | Low
"maximumFee": "5000"   // hard cap in drops

// Ripple Custody, option B — base network fee + a fixed bump
"feeStrategy": { "type": "SpecifiedAdditionalFee", "drops": "20" },
"maximumFee": "5000"

// Palisade — fee is mostly managed; the transfer op takes an optional cap
// (decimal XRP, not drops), and other XRP Submit* ops expose no fee field at all
"networkFee": "0.0002"
```

How the SDK maps fee. It exposes one optional, normalized fee intent, never raw drops, and translates it per path: `fee?: { priority?: "low" | "medium" | "high"; maxFeeDrops?: string }`.

- *Omitted* → each backend's managed default (xrpl.js autofill for Local/raw; `Priority: Low` on Custody (matching custody.js's `options.feePriority ?? "Low"`); managed on Palisade).
- `priority` → Custody `feeStrategy: Priority`; Local/raw scale the autofilled fee; Palisade has no way to pass a priority at submission, so the SDK lets Palisade auto-price the fee and warns that `priority` isn't applied there.
- `maxFeeDrops` (the cap both honor) → Custody `maximumFee`; Palisade `networkFee` (drops→XRP decimal); Local/raw bound xrpl.js autofill (`maxFeeXRP`) and throw `IntentValidationError` if exceeded.

The cap is the common contract across all paths; the SDK never fabricates an exact fee on native custody paths — it forwards the intent and lets the custodian price it (§5.3).

Callers don't set fees every call. The default intent is a per-custodian `defaultFee` set once at construction (§3.3, §3.4), with a built-in fallback of `Priority: Low` plus a safety max-fee cap, overridable per call. So although Custody's API *requires* a `feeStrategy` on every order, the SDK supplies it; the end user passes `fee` only to override.

7.5 Approval summary (`customProperties`)

Custody exposes a free-form metadata bag on each intent that it stores and displays to approvers but does **not** validate or interpret — there is no custodian-side schema. The field is **required by the Custody API on every intent** — both RippleNative and RippleRaw, since they share one signed-intent envelope; it may be `{}`, but the SDK always populates it. `simpleXRPL` generates one during intent construction: a small, human-readable block (verb, amount, destination) so approvers see meaningful text instead of a raw transactor (on the RippleRaw path it's the only readable surface an approver gets — the rest is a base64 blob). **Palisade has no equivalent field today, so nothing is sent on PalisadeNative or PalisadeRaw.**

8. Idempotency & Retries

A submission creates a funds-moving intent, so the SDK must never silently send one twice. Design choices:

- **The SDK does not retry on its own.** A failed write surfaces to the caller, who decides whether to retry.
- **Retries are safe by design.** Each submission carries a stable, client-generated id — a time-ordered **UUIDv7** (as custody.js xrpl.builders.ts uses for `requestId` / `payloadId`; time-ordering keeps ids sortable and index-friendly) mapped to Custody's intent UUID / Palisade's `externalId` — surfaced on the result and on `IntentPendingError`. Retrying with the same id resolves to the same intent instead of creating a duplicate. (Local transactions dedupe on-ledger via `Sequence`, so they need no extra key.)
- **Multi-step verbs are the exception to "all or nothing".** Ordered verbs (e.g. `IOU.issue`) commit each step before the next, so a partial failure throws `MultiStepFailureError` listing which steps committed (§11) with no rollback. To finish, the caller re-runs only the remaining step with the matching single-step verb (e.g. `IOU.transfer` for the final `Payment` of `IOU.issue`), not the whole verb.

9. Session, Identity, Key Management

Each custodian carries its own credentials and its own notion of identity. `simpleXRPL` holds no global session: the active custodian for a call is resolved at dispatch time from the `Account` being acted on.

9.1 What `SimpleXRPL.init(...)` ties to

`SimpleXRPL.init()` binds the client to a network (rippled URL, network id, optional faucet URL) and to a set of pre-constructed `Custodian` instances (see the call shape in §3.1). Each custodian was already authenticated at its own `create()` / `fromEnv()` call, and brought its own primary account and discovered account list with it. Example:

```
const ripple = await RippleCustody.fromEnv({ primary: "rTreasury..." });
const palisade = await PalisadeCustody.fromEnv({
  primary: { vaultId: "v_...", walletId: "w_..." },
});

const client = await SimpleXRPL.init({
  rippledUrl: TESTNET_WS,
  signers: [ripple, palisade],
  // primarySigner defaults to signers[0]; explicit override:
  primarySigner: ripple,
});
```

Account binding is implicit: every account each custodian discovered is registered against that custodian. The same r-address appearing under two custodians is rejected at `init` (§11 `AmbiguousAccountError`); the caller must drop one or supply an explicit per-call override.

There is no notion of "the current session's key." The dispatch rule always resolves the custodian through `account.custodian` at call time.

9.2 Account discovery and refresh

- Each custodian populates its account cache during `create()` / `fromEnv()`:
  - `RippleCustody` paginates `GET /v1/domains/{domainId}/accounts` (limit 100) and keeps the full list.
  - `PalisadeCustody` lists vaults, then wallets per vault, filtered to the active XRPL ledger.
  - `LocalSigner` enumerates its `xrpl.js` `Wallet`s.
- `client.refreshAccounts()` invokes `listAccounts()` on every registered custodian and rebuilds the client-side address→custodian index. New accounts become addressable; accounts removed upstream become `AccountNotFoundError` on next use.

9.3 No-signer initialization

Reads never require a signer — they go through `xrpl.js` directly to rippled. A `SimpleXRPLClient` constructed with no signers is therefore fully functional for queries (`*.balance`, `account.info`, ledger lookups, etc.); only write methods throw `NoSignerError` until a signer is added. This makes simpleXRPL usable for exploration and tests without forcing a credential setup decision.

If `signers` is omitted but `XRPL_*_SEED` variables are present, `LocalSigner.fromEnv()` is constructed automatically and used as the single custodian.

9.4 Cross-account flows (e.g. IOU issuance)

IOU issuance needs an issuer account *and* a holder account. Both are addressed by r-address:

```
await client.IOU.issue({
  issuer: "rIssuer...",
  holder: "rHolder...",
  ticker: "USD",
  amount: "1000",
});
```

`IOU.issue` expands into an ordered sequence of sub-transactions (`AccountSet` + `TrustSet` + `Payment`). Each step is built, validated, and dispatched independently against the account that signs it; the orchestrator commits each step before starting the next, and partial-failure semantics are covered in §8 and §11.

- **Both accounts on the same `RippleCustody` .** No re-authentication between steps: one intent-author logs in once (the JWT lasts ~4 hours) and that session covers every step. Each intent names the account it acts on, and Custody's policy engine checks the caller is allowed to act on it.
- **Accounts on different custodians** (e.g. issuer on `RippleCustody` , holder on `LocalSigner` , or two different Custody domains). Each step goes to whichever custodian owns that account. The SDK runs the steps in order but does not link them; there is no shared session and no cross-custodian atomicity, so if a later step fails the earlier ones stay committed

9.5 Token lifetime and refresh

- `RippleCustody` caches the **JWT** bearer token — a short-lived signed token (here ~4 h) attached to every API call as proof of an authenticated session — and refreshes it transparently before expiry (expiry read from the JWT `exp` claim with a ~5-min safety buffer, 4-hour fallback). It obtains the token by **challenge-response**: it signs a one-time random nonce (the *challenge*) with the intent-author key to prove it holds the key, and the server returns the JWT — flow defined in `products/custody/@v1.35/concepts/authenticate-api-requests.md` . Three refinements (proven in custody.js auth.service.ts/ api.service.ts): concurrent callers share a **single in-flight refresh**; each refresh signs a **fresh challenge**; and a `401` triggers one **refresh-and-retry** with a new challenge. Refresh failures surface as `CustodyAuthError` .
- `PalisadeCustody` uses long-lived API keys; rotation is the caller's responsibility. Auth failures surface as `PalisadeAuthError` .

- Beyond that single 401 refresh-retry, simpleXRPL does not auto-retry auth or policy failures; the caller decides.

10. Asynchronous Execution Model

`simpleXRPL` exposes two execution modes for write methods. Both are `async` and produce the same on-ledger / custodian-side outcome; they differ in **what the returned Promise resolves with and when**:

- `submitAndWait` **(default, §10.1)** — the Promise resolves with the final `SubmissionResult` once the transaction reaches a terminal state, or throws `IntentPendingError` if the configured `defaultTimeoutMs` elapses first. Mirrors `xrpl.js` `submitAndWait()`. Best for short-running flows where the caller wants the result inline.
- `submitAsync` **(opt-in, §10.2)** — the Promise resolves as soon as the custodian has accepted the intent and returned an ID, handing the caller a `SubmissionHandle` to poll or wait on later. The intent's lifetime is decoupled from the caller's request lifetime. Best for governance flows that may span hours or days, or for processes that should not sit blocked.

Mechanically the sync path is the async path plus an immediate `handle.wait(defaultTimeoutMs)` with the handle discarded; in both modes the timeout-recovery story is identical (catch `IntentPendingError`, resume via `client.intent.await(...)`).

10.1 Default: `submitAndWait`-style

Every write method is `async` and blocks until the transaction reaches a terminal state. Terminal definitions per path:

Path	Terminal success	Terminal failure	Timeout behavior
Local	`xrpl.js` returns `meta.TransactionResult == "tesSUCCESS"` from `submitAndWait`.	`tem*`, `tec*`, `tef*`, `tel*`	Bounded by `LastLedgerSequence` (`xrpl.js` default); call throws on exceed.
RippleNative	Intent reaches `Executed` and the wrapped transaction is confirmed on-ledger.	Intent `Rejected` or `Expired` (custodian-side `expiryAt`	Bounded by `RippleCustodyConfig.defaultTimeoutMs` (default 60s (custody.js default is 30s); cold-

		elapsed), or transaction `Failed`.	vault workflows can override per-call). On timeout the call throws `IntentPendingError` carrying the `intentId` (non-terminal); the intent keeps living custodian-side until approved, executed, or `Expired`.
RippleRaw	After Custody returns the signed blob, the rippled `submitAndWait` outcome applies (same as Local).	Auth failure, policy rejection, or rippled-side failure (same as Local).	Custody portion uses `RippleCustodyConfig.defaultTimeoutMs`; rippled portion uses `LastLedgerSequence`.
PalisadeNative	The native endpoint returns a confirmed `txHash` (and Palisade reports the transaction as terminal).	Policy rejection, Palisade API error, or rippled-side failure.	Bounded by `PalisadeCustodyConfig.defaultTimeoutMs`.
PalisadeRaw	After Palisade returns the signed blob, the rippled `submitAndWait` outcome applies (same as Local).	API-key failure, policy rejection, or rippled-side failure.	Palisade portion uses `PalisadeCustodyConfig.defaultTimeoutMs`; rippled portion uses `LastLedgerSequence`.

Intent lifetime vs. SDK timeout - two different clocks. `defaultTimeoutMs` is how long the *SDK* waits before handing control back (throwing `IntentPendingError`); it does not end the intent. The custodian intent carries its own `expiryAt` — a server-side lifetime (default ~1 day, overridable per call / at init) — and if approvers don't act before it, the intent becomes the terminal status `Expired`. So a governance intent can outlive the SDK's wait (→ `IntentPendingError`, resumable via §10.4) and *then* expire custodian-side; a resumed `client.intent.await` later observes `Expired` as a terminal failure. Palisade's analog is the approval-group timeout (§10.4), which auto-rejects.

10.2 Opt-in: `submitAsync`

For any long-running approval cycle, the caller can opt in by passing `{ async: true }`:

```ts
const handle = await client.token.transfer({
  to: "rDest...",
  amount,
  async: true,
});
// handle: { kind, id, custodian, poll(), wait(timeoutMs?), cancel?() }

const status = await handle.poll(); // non-blocking snapshot
const result = await handle.wait(); // no-arg: bounded wait, defaults to defaultTimeoutMs (60 s)
const longResult = await handle.wait(24 * 3600_000); // override: block up to 24h
```

`SubmissionHandle.id` is the custodian-native id (Custody `intentId`, Palisade `transactionId`) or the XRPL `txHash` for Local. `poll()` and `wait()` return the same `SubmissionResult` shape as the sync variant once terminal.

`wait(timeoutMs?)` with no argument defaults to the custodian's `defaultTimeoutMs` (60 s) - a *bounded* wait, not a long block. For genuine cold-vault / governance waits, either pass an explicit long `timeoutMs` (e.g. `handle.wait(24 * 3600_000)`) or — preferred — persist the `intentId` and resume later via `client.intent.await`, so no process sits blocked for hours. On timeout it throws `IntentPendingError` carrying the `intentId`.

10.3 Cancellation

- Local: no cancellation once submitted; rejected with a clear error if called.
- RippleNative: `cancel()` posts a Custody cancel intent if the underlying intent is still in an approvable state; otherwise rejects.
- Palisade native: `cancel()` calls the Palisade cancel endpoint if the transaction is still pending approval; otherwise rejects.
- Raw-signing paths (RippleRaw, PalisadeRaw) between custodian-sign and rippled-submit: not meaningful in practice (the signed blob already exists); the API rejects.

10.4 Governance multi-approval

Both Ripple Custody and Palisade can be configured for M-of-N human approval of an intent before the ledger-signing key is released. simpleXRPL is a **proposer**, not an approval coordinator: it submits an intent and observes its terminalization. The approvers interact with the custodian's own UI / API; simpleXRPL has no view into who they are or what they decide, only the final outcome.

In both custodian, multi-party control is enforced off-ledger by the custodian's policy engine while the on-ledger transaction stays single-signature, so no protocol multisign is needed. The

two products express it slightly differently:

Ripple Custody. Every submitted intent is evaluated against ranked **policies**, each with a workflow of approval steps naming an approval group (users with a role) that must approve, chainable with `AND` / `OR`; the vault key is released only once the workflow is satisfied. simpleXRPL polls the outcome via `GET /v1/intents/{id}` and never touches policy or approver configuration.

Palisade. An **approval group** lists eligible approvers and a minimum required count per transaction (the literal M-of-N), with a configurable timeout (default 1 h) that auto-rejects if unmet. simpleXRPL polls the outcome via `TransactionsService_GetTransaction` and leaves group and policy-rule configuration to the Palisade console.

11. Error Model

Errors are pass-through but typed. simpleXRPL does not flatten distinct underlying failure modes into a single class.
A few notes:

- Custodian errors preserve `processing.hint` (Custody) or the equivalent Palisade diagnostic field verbatim. The caller decides whether to surface them.
- Rippled errors preserve `engineResult` verbatim.
- `IntentPendingError` is **not a failure**; it is a "still waiting" signal that lets the synchronous code path return without holding the process. Resume with the handle's `intentId`.
- Multi-step verbs (e.g. `IOU.issue`) throw `MultiStepFailureError` on partial failure. simpleXRPL does **not** attempt automatic rollback. The error carries the list of already-committed steps so the caller (or an operator) can reconcile manually.

```ts
class SimpleXRPLError extends Error {
  /* base */
}

class IntentValidationError extends SimpleXRPLError {}
class SignerCapabilityError extends SimpleXRPLError {}

class NoSignerError extends SimpleXRPLError {}
class AccountNotFoundError extends SimpleXRPLError {
  readonly account: string;
}
class AmbiguousAccountError extends SimpleXRPLError {
  readonly account: string;
  readonly custodians: ReadonlyArray<
    "local" | "ripple-custody" | "palisade-custody"
  >;
}

// Custodian-side errors
class CustodyAuthError extends SimpleXRPLError {}
class CustodyApiError extends SimpleXRPLError {
```

```
  readonly status: number;
  readonly hint?: string; // processing.hint, preserved verbatim
  readonly raw: unknown; // full Custody response body
}
class PalisadeAuthError extends SimpleXRPLError {}
class PalisadeApiError extends SimpleXRPLError {
  readonly status: number;
  readonly raw: unknown;
}

class IntentPendingError extends SimpleXRPLError {
  readonly intentId: string;
  readonly custodian: "ripple-custody" | "palisade-custody";
  readonly lastState: string;
}

class RippledSubmitError extends SimpleXRPLError {
  readonly engineResult: string; // e.g. "tecPATH_DRY"
  readonly raw: unknown; // full rippled response
}

class MultiStepFailureError extends SimpleXRPLError {
  readonly committed: SubmissionResult[]; // sub-transactions that succeeded
  readonly failed: { step: number; error: SimpleXRPLError };
}
```

12. Security Considerations

12.1 Raw-signing opt-in

Both `RippleCustody` and `PalisadeCustody` default to `allowRawSigning: false`. The dispatcher throws `SignerCapabilityError` at the call site for any verb that would otherwise dispatch via a raw-signing path (RippleRaw; Palisade `RawTransaction` / `SignPlaintext`). The error message states both the local config change and the operator-side requirement, e.g. for Custody:

1. `allowRawSigning: true` in `RippleCustodyConfig`, and
2. `HMZ_NOTARY_ENABLE_UNSAFE_SIGN_MANIFESTS=TRUE` on the Custody governance engine.

12.2 Credential handling & key storage

- **The SDK persists nothing to disk.** The intent-author signing key is caller-supplied and held in memory for the process lifetime; the Custody JWT is *derived* from it via challenge-response (§9.5) and cached in memory — in a long-lived service it self-refreshes, in a one-shot process it is re-derived per run. Neither the key nor the token is ever written to disk, and both are redacted from logs (§3.5). So the SDK adds no on-disk credential footprint of its own — same posture as `xrpl.js` with a seed.
- **Prefer a KMS / secret manager (the recommended default).** The signing key (Custody) and API key (Palisade) should be fetched at runtime from a secret manager (Vault, AWS/GCP Secrets Manager, …) rather than left on disk. The `.env` / `.pem`-path / `XRPL_*_SEED`

conveniences (§3.2–§3.4) are for development: files on disk are a supply-chain exfiltration target (e.g. recent npm `Shai-Hulud`-class attacks that scrape `.env` / `.pem`).

- **Bounded blast radius.** The Custody signing key can only *propose* intents — Custody's M-of-N governance (§10.4) gates execution — so a compromised signing key cannot move funds on its own; the worst case is an attacker submitting a proposal that approvers must still sign off. (A `LocalSigner` seed, by contrast, signs fund-moving transactions directly, which is why Local is for development/utility accounts and production accounts should use a governed custodian.)
- **HSM / KMS-backed signing is a future, pluggable signer.** v1 signs in-process. The `Custodian` interface (§4) is the seam for an HSM/KMS-backed signer where the private key never leaves the secure boundary; it can be added in v2 without a breaking change. Deferred for v1 given the bounded blast radius above and parity with `xrpl.js` / `xrpl-py` (which also sign in-process).12.3 Transport
- All custodian traffic is HTTPS; the client rejects plain-HTTP base URLs.
- TLS certificate pinning is not implemented in v1.

12.4 Governance boundary

simpleXRPL never participates in custodian governance. It does not expose approve/reject endpoints, approver identity, quorum configuration, or audit-log reads. Any caller that needs those capabilities should integrate the custodian's own UI or admin API directly.

13. Testing & Documentation

Strategy and tooling only; specific test cases live in a separate test plan, and the per-verb doc content is generated from code.

13.1 Testing

Tooling matches xrpl.js: **Jest** with separate `jest.config.unit.js` / `jest.config.integration.js` configs, plus **TypeScript** `tsc` for public-surface type-checking. Two tiers:

Offline (no network):

- **Unit**: pure Build/Validate functions: alias resolver, amount/scale math, flag-combination matrix, and the `customProperties` generator. ≥85% line coverage.
- **Mock**: dispatch + `Custodian` orchestration with the network stubbed via the adapters' injected I/O ports (§7.1) — in-memory ports replace HTTP, so mapping and routing run offline.

Asserts every config permutation routes correctly.**Live —** (real network, run serially with `--runInBand` to avoid sequence-number conflicts on shared accounts):

- **Integration (testnet)** — Local end-to-end plus RippleRaw and Palisade raw; runs each verb and asserts the discriminated-union shape.
- **Contract (custodian sandboxes)** — one per native, emits drift alerts when Custody/Palisade change shape. Bumping the pinned custodian API version must keep the previous version's suite green.

13.2 Documentation

- **Generated reference**: **TypeDoc** from JSDoc on the public API. Type signatures are the spec, prose adds context; every public class, method, and interface carries a JSDoc block.
- **Guides**: getting-started per signer (Local, Ripple Custody, Palisade); mixed-custodian cookbook; long-running approvals; troubleshooting.
- **Dispatch-table mapping doc**: in the README and docs site, updated whenever the table changes.
- **When simpleXRPL isn't enough**: points to `xrpl.js` for protocol-level access.
- **Code Snippets:** requires self-contained, runnable examples per verb showing the SDK call and the transactor it maps to. Use standalone code snippets, not the integration tests.

14. Implementation Plan

1. **Scaffolding & domain model.** Package setup, the `Custodian` interface, core types (`Account`, `SignerCapabilities`, `SubmissionResult`), error classes, and `SimpleXRPL.init` construction including no-signer mode.
2. **Walking skeleton — Local, one verb.** `LocalSigner` plus the full pipeline for a single verb (e.g. `XRP.transfer`) end-to-end on testnet.
3. **Vertical breadth on Local.** Fill out the remaining verticals against `LocalSigner`, including the multi-step orchestrator (see Section 8).
4. **RippleCustody adapter.** Auth, account discovery, native intent construction + `customProperties`, the RippleRaw path, and governance observation / async (`intent.status` / `await`, `submitAsync`). Contract tests vs sandbox.
5. **PalisadeCustody adapter.** API-key auth, vault/wallet discovery, native `Submit*` mapping, raw path. Contract tests vs sandbox.
6. **Mixed-signer + hardening.** Per-account dispatch across custodians, idempotency, logging/redaction, then the docs surface.